

Recursive Mutexes in Separation Logic

Ke Du

kdu9@uic.edu

University of Illinois Chicago
Chicago, Illinois, USA

Paolo G. Giarrusso

paolo@skylabs-ai.com

Skylabs AI

William Mansky

mansky1@uic.edu

University of Illinois Chicago
Chicago, Illinois, USA

Gregory Malecha

gregory@skylabs-ai.com

Skylabs AI

Abstract

Mutexes (i.e., locks) are well understood in separation logic, and can be specified in terms of either protecting an invariant or atomically changing the state of the lock. In this abstract, we develop the same styles of specifications for *recursive* mutexes, a common variant of mutexes in object-oriented languages like C++ and Java. A recursive mutex can be acquired any number of times by the same thread, and our specifications treat all acquires/releases uniformly, with clients only needing to determine whether they hold the mutex when accessing the lock invariant.

CCS Concepts: • Theory of computation → Program verification.

Keywords: concurrent separation logic, C++

ACM Reference Format:

Ke Du, William Mansky, Paolo G. Giarrusso, and Gregory Malecha. 2018. Recursive Mutexes in Separation Logic. In *Proceedings of Rocq for Programming Languages (RocqPL '26)*. ACM, New York, NY, USA, 3 pages. <https://doi.org/XXXXXX.XXXXXX>

1 Introduction

By now, mechanized concurrent separation logic can prove correct programs in real languages, using tools like VST [2], BRiCk [1], or RustBelt [4]. To enable verifying more programs, specifying and verifying standard libraries (like ARC in Rust) has high leverage. Here, we report our progress on verifying C++’s concurrency library [3] using BRiCk. While some utilities are standard, others have not been previously analyzed in separation logic. In particular, we address *recursive mutexes*: a thread owning a non-recursive mutex

cannot acquire the same mutex again, while a thread owning a recursive mutex can acquire it again, as shown in fig. 1. Crucially, methods `update_a` and `transfer` are both thread-safe, and `transfer` can reuse `update_a` even if both acquire the same mutex! With normal mutexes, `transfer` couldn’t call `update_a`; typically, `transfer` could only call an alternative `update_a_no_lock`. To sum up, acquiring a recursive mutex is not just more convenient, but more transparent and better encapsulated than using a normal mutex.

```
class C {  
    int balance_a; int balance_b;  
    std::recursive_mutex mut;  
public:  
    void update_a(int x) {  
        mut.lock(); balance_a += x; mut.unlock();  
    }  
    void update_b(int x) {  
        mut.lock(); balance_b += x; mut.unlock();  
    }  
  
    void transfer(int x) {  
        mut.lock(); update_a(x); update_b(-x); mut.unlock();  
    }  
};
```

Figure 1. An example using recursive mutexes.

In this abstract we give a separation logic specification for recursive mutexes, and show that it allows encapsulation reasoning: in particular, a client can acquire a recursive mutex and access its protected resources without doing case analysis on whether they already own the mutex.

2 Specifying the recursive mutex

The standard separation logic specification for a mutex/lock associates a lock ℓ with resources P (the “lock invariant”), represented as $\text{is_lock}(\ell, P)$. The acquire and release functions are then specified by:

$$\{\text{is_lock}(\ell, P)\} \text{lock}(\ell) \{\text{is_lock}(\ell, P) * P\}$$

$$\{\text{is_lock}(\ell, P) * P\} \text{unlock}(\ell) \{\text{is_lock}(\ell, P)\}$$

A thread that acquires the lock gains access to the protected resources, and gives them up upon release. Our goal is to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
RocqPL '26, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06...\$15.00

<https://doi.org/XXXXXX.XXXXXX>

state similar specifications for recursive mutexes. The key points are:

- A fractional *handle* $\ell \mapsto_q \text{rmutex } \gamma P$ that is held by each thread that knows about the recursive mutex γ with invariant P .
- A predicate $\text{holds } \gamma t n P$ indicating that thread t has acquired mutex γ with invariant P n times. When n is 0, t does not hold the mutex; when n is greater than 0, it does, and can borrow P from $\text{holds } \gamma t n P$.

The main difference from the simple lock is that every thread that might interact with mutex γ , whether it currently holds it or not, has its own $\text{holds } \gamma t n P$ assertion; n is simply 0 for any thread that does not currently hold the mutex. The mutual exclusion property is reflected by the fact that $\text{holds } \gamma t_1 n_1 P * \text{holds } \gamma t_2 n_2 P \vdash n_1 = 0 \vee n_2 = 0$, i.e., at most one thread holds the mutex at a time. Furthermore, it is vital that there is only one holds predicate per thread; otherwise, a thread can lock the same lock twice with different instances of holds and obtain $\text{holds } \gamma t 1 P * \text{holds } \gamma t 1 P$, therefore capable of borrowing 2 copies of P . Thus, the holds predicate must also satisfy $\text{holds } \gamma t n_1 P * \text{holds } \gamma t n_2 P \vdash \text{False}$.

From a programmer's perspective, the key feature of a recursive mutex is that we hold the protected resources after acquiring it, whether or not we held them before. Thus, our specs should let us conclude that we own the invariant after a call to lock without requiring case analysis on whether we already hold the lock. Our specifications allow this by simply saying that lock increases n and unlock decreases it:

$$\begin{aligned} & \{ \ell \mapsto \text{rmutex } \gamma q * \text{holds } \gamma t n P \} \\ & \quad \text{lock}(\ell) \\ & \{ \ell \mapsto \text{rmutex } \gamma q * \text{holds } \gamma t (n+1) P \} \\ & \{ \ell \mapsto \text{rmutex } \gamma q * \text{holds } \gamma t (n+1) P \} \\ & \quad \text{unlock}(\ell) \\ & \{ \ell \mapsto \text{rmutex } \gamma q * \text{holds } \gamma t n P \} \end{aligned}$$

These specifications implicitly say that if n was 0 before a lock, then afterwards t gains ownership of P , and if n is 0 after an unlock, then t gave up ownership of P . However, these are not special cases in the specs—instead, threads may freely acquire and release ℓ , incrementing and decrementing their counts n , and then extract P from holds whenever they can show that n is greater than 0. This allows proofs that follow the standard use pattern of recursive mutexes: whenever a thread needs access to P , it first acquires the mutex regardless of whether it already held it, and then releases it once finished, restoring the count to the value before the process.

The final piece is the “borrowing” rule that allows a thread to obtain the invariant P if it holds the lock:

$$\begin{aligned} n > 0 & \Rightarrow \ell \mapsto \text{rmutex } \gamma q * \text{holds } \gamma t n P \Leftrightarrow \\ & \ell \mapsto \text{rmutex } \gamma q * P * \text{held } \gamma t n \end{aligned}$$

A thread can exchange its positive holds predicate for ownership of P , plus a retainer held that tracks n but does not give access to P . This exchange operation is a view shift because it is implemented via an Iris invariant [5], but this is mostly irrelevant from the user's perspective. Essentially, while the standard lock acquire spec simultaneously acquires the lock and gives the thread ownership of P , our recursive mutex specs separate these operations into two steps, incrementing n on acquire and then letting the thread obtain P at any later point, as long as n is greater than 0. Since holds is part of the precondition of unlock, the thread must unborrow P before releasing the lock, whether or not the release actually sets n to 0.

3 Fixing Arguments

Lock invariants are usually written with existentially quantified arguments: for instance, a lock that protects the location x may have the invariant $P \triangleq \exists v. x \mapsto v$. In this case, the borrowing rule's requirement that return P to holds before acquiring or releasing the mutex effectively re-quantifies the variable, forcing the thread to lose information about resources it owns. We fix this by packing any such arguments into a tuple a , provided as an additional parameter to holds: $\text{holds } \gamma t n P a$ means that if $n > 0$, then we hold resources $P(a)$, and the argument a will remain the same as long as $n > 0$. We use this approach to verify the example from fig. 1.

The invariant for the example is

$$P(a, b) \triangleq \text{balance_a} \mapsto a * \text{balance_b} \mapsto b$$

Our specification for `update_a` is then:

$$\begin{aligned} & \{ \text{mut} \mapsto \text{rmutex } \gamma q * \text{holds } \gamma t n P(a, b) \} \\ & \quad \text{update_a}(x) \\ & \{ \text{mut} \mapsto \text{rmutex } \gamma q * \text{holds } \gamma t n P(a+x, b) \} \end{aligned}$$

and similarly for `update_b`. When a thread calls `update_a`, it gets back the lock with the same n as before the call; if that n is greater than 0 then we know that the new value of `balance_a` is $a+x$, while otherwise we only know that the invariant $\exists a, b. P(a, b)$ has been preserved. Since transfer begins by acquiring the mutex, we can show that the final value of `balance_a` + `balance_b` is the same as the initial value.

4 Discussion

We have seen that the recursive mutex specs let us support the common pattern of functions that work equally well whether or not the caller holds the lock before calling, without losing any information about the invariant upon return. In our implementation, the specs are derived from more primitive *logically atomic* specs and an associated invariant, but these details are entirely hidden from the client. This work is part of our broader effort to verify the C++ standard library; we also have finished or in-progress work on ...

References

- [1] 2025. The BRiCk Project. <https://github.com/SkylabsAI/BRiCk>.
- [2] Andrew W. Appel, Robert Dockins, Aquinas Hobor, Lennart Beringer, Josiah Dodds, Gordon Stewart, Sandrine Blazy, and Xavier Leroy. 2014. *Program Logics for Certified Compilers*. Cambridge University Press. <http://www.cambridge.org/de/academic/subjects/computer-science/programming-languages-and-applied-logic/program-logics-certified-compilers?format=HB>
- [3] ISO. 2024. *ISO/IEC 14882:2024 Programming languages — C++* (seventh ed.). International Organization for Standardization, Geneva, Switzerland. 2104 pages. <https://www.iso.org/standard/83626.html>
- [4] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2017. RustBelt: Securing the Foundations of the Rust Programming Language. *Proc. ACM Program. Lang.* 2, POPL, Article 66 (Dec 2017), 34 pages. <https://doi.org/10.1145/3158154>
- [5] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. ACM, 637–650. <https://doi.org/10.1145/2676726.2676980>